

CHECKLIST DE ESTRATEGIA

Calidad de Software Completa

1

Documentos Obligatorios del Proyecto

Qué documentos crear, quién los crea y cómo estructurarlos

1.1 Test Plan (Plan de Pruebas) - Cumplir con todos los puntos

El documento más importante del proceso de calidad. Define el alcance, enfoque, recursos y calendario de todas las actividades de prueba.

Estructura requerida del Test Plan:

- 1. Introducción y objetivos del plan de prueba
- 2. Alcance: qué se prueba y qué está fuera de alcance
- 3. Estrategia de prueba por capa (Unit → Integration → E2E → Performance → Security)
- 4. Tipos de prueba a ejecutar y justificación
- 5. Criterios de entrada (cuándo empezar a probar)
- 6. Criterios de salida (cuándo dejar de probar / cuándo el software está listo)
- 7. Criterios de suspensión (cuándo pausar si algo sale mal)
- 8. Roles y responsabilidades del equipo de QA
- 9. Entornos de prueba: local, staging, producción
- 10. Herramientas utilizadas con justificación
- 11. Gestión de defectos: flujo, severidades y prioridades
- 12. Métricas de calidad objetivo (cobertura, tasa de defectos, etc.)
- 13. Riesgos identificados y plan de mitigación

	Ítem	Responsable	Estado
☐	Test Plan creado y aprobado por el equipo	QA Lead	
☐	El plan incluye las cuatro capas: frontend, API, BD relacional, BD NoSQL	QA Lead	
☐	Los criterios de entrada y salida son medibles y específicos	QA Lead	
☐	Entornos de prueba documentados con diferencias entre ellos	DevOps	
☐	El Test Plan está versionado en el repositorio del proyecto	Equipo	

1.2 Casos de Prueba - Cumplir con al menos 5 puntos

Documentación detallada de cada escenario de prueba. Cada caso debe ser reproducible por cualquier miembro del equipo sin conocimiento previo del sistema.

Campos obligatorios por caso de prueba:

	Ítem	Responsable	Estado
☐	ID único del caso (ej: TC-API-001, TC-UI-023)	QA	
☐	Título descriptivo: módulo + acción + resultado esperado	QA	
☐	Precondiciones: estado del sistema antes de ejecutar	QA	
☐	Datos de entrada específicos (no 'datos válidos', sino los datos reales)	QA	
☐	Pasos de ejecución numerados y granulares	QA	
☐	Resultado esperado concreto y verificable	QA	
☐	Resultado real (completar al ejecutar)	QA	
☐	Estado: Pass / Fail / Blocked / Not Executed	QA	
☐	Severidad y prioridad del caso	QA	
☐	Automatizado: Sí / No / Pendiente	QA	
☐	Referencia al requerimiento o historia de usuario	QA	

1.3 Registro de Defectos (Bug Report) - Cumplir con al menos 5 puntos

Campos obligatorios por defecto reportado:

	Ítem	Responsable	Estado
☐	ID único del defecto (ej: BUG-142)	QA	
☐	Título: qué falla, dónde y bajo qué condición	QA	
☐	Descripción: comportamiento actual vs esperado	QA	
☐	Pasos exactos para reproducir (incluyendo datos de prueba usados)	QA	
☐	Evidencia: capturas, logs, videos, respuestas HTTP	QA	
☐	Severidad: Critical / High / Medium / Low	QA	
☐	Prioridad: Blocker / High / Medium / Low	QA	
☐	Entorno donde se encontró (OS, browser, versión, ambiente)	QA	
☐	Versión del sistema donde se encontró	QA	
☐	Asignado a: desarrollador responsable	QA Lead	
☐	Estado del ciclo: Open → In Progress → Fixed → Verified → Closed	Dev/QA	

2

Configuración del Entorno de Calidad

Estructura de proyecto, directorios, configuración de herramientas y CI/CD

2.2 Configuración TypeScript para calidad - Cumplir con todos los puntos

	Ítem	Responsable	Estado
☐	tsconfig.json con strict: true activado	Dev	
☐	noImplicitAny: true — sin tipos implícitos any	Dev	
☐	strictNullChecks: true — manejo explícito de null/undefined	Dev	
☐	noUnusedLocals y noUnusedParameters: true	Dev	
☐	ESLint con @typescript-eslint/recommended configurado	Dev	
☐	Prettier integrado con ESLint para formato consistente	Dev	
☐	Regla de no-explicit-any activa en ESLint	Dev	

2.3 Cobertura mínima sugerida

Capa de dominio (lógica de negocio)		90%
Capa de aplicación (servicios/casos de uso)		80%
Capa de presentación (controllers)		70%
Capa de infraestructura (repositorios)		60%
Cobertura global mínima del proyecto		75%

3

Pruebas Unitarias

Vitest + @testing-library · Dominio, servicios, utilidades y componentes UI

3.1 Qué probar con unit tests - Cumplir con todos los puntos

Regla de oro: un unit test prueba UNA SOLA unidad de código en completo aislamiento.**Todo lo externo (BD, HTTP, sistema de archivos, tiempo) debe ser mockeado.****Unidades que DEBEN tener unit tests: - Cumplir con**

	Ítem	Responsable	Estado
☐	Funciones de lógica de negocio pura (cálculos, validaciones, transformaciones)	Dev	

3.2 Buenas prácticas de unit testing en TypeScript - Cumplir con todos los puntos

	Ítem	Responsable	Estado
☐	Cada test sigue el patrón Arrange-Act-Assert (AAA) explícitamente	Dev	
☐	Nombres descriptivos: describe('UserService') > describe('cuándo el email ya existe') > it('debe lanzar error DuplicateEmailError')	Dev	
☐	Un test, una sola aserción (o un solo concepto por aserción)	Dev	
☐	No hay lógica de if/else ni bucles dentro de los tests	Dev	
☐	Tests independientes entre sí: sin dependencia del orden de ejecución	Dev	

4

Pruebas de Integración

Supertest · Testcontainers · BD Relacional + NoSQL · API REST

4.1 Pruebas de la API REST

Checklist por endpoint de la API: - Cumplir con al menos 5 puntos

	Ítem	Responsable	Estado
☐	Happy path: request válido retorna status code correcto (200/201/204)	Dev	

	Ítem	Responsable	Estado
☐	Schema de respuesta: todos los campos esperados presentes con tipos correctos	Dev	
☐	Validación de entrada: campos requeridos faltantes → 400 Bad Request	Dev	
☐	Validación de tipos: tipo incorrecto en el body → 400 con mensaje descriptivo	Dev	
☐	Autenticación: request sin token → 401 Unauthorized	Dev	
☐	Autorización: token sin permisos suficientes → 403 Forbidden	Dev	
☐	Recurso inexistente: ID que no existe → 404 Not Found	Dev	
☐	Conflicto de datos: recurso duplicado → 409 Conflict	Dev	
☐	Límites de payload: body muy grande → 413 o error apropiado	Dev	
☐	Headers de respuesta correctos: Content-Type, CORS, Cache-Control	Dev	
☐	Paginación: page, limit, total, hasNextPage en respuestas de colecciones	Dev	
☐	Filtros y búsqueda: query params aplicados correctamente	Dev	
☐	Ordenamiento: orden ascendente y descendente funcionan	Dev	
☐	Idempotencia: PUT/DELETE aplicados dos veces dan el mismo resultado	Dev	

4.2 Pruebas de Base de Datos Relacional (PostgreSQL/MySQL) - Cumplir con al menos 3 puntos

	Ítem	Responsable	Estado
☐	Testcontainers configurado para levantar BD real en Docker durante los tests	Dev	
☐	Migraciones se ejecutan correctamente en la BD de test antes de los tests	Dev	
☐	afterEach o afterAll hace rollback o truncate de tablas para aislamiento	Dev	
☐	Tests de repositorios: CRUD completo con datos reales en BD	Dev	
☐	Constraints de BD: UNIQUE, NOT NULL, FK violations probadas	Dev	
☐	Transacciones: rollback ante error en operaciones complejas	Dev	
☐	Índices: queries con grandes volúmenes no generan full scan	Dev	
☐	N+1 queries: verificado con query logger en entorno de test	Dev	
☐	Seeds de datos de prueba son deterministas (mismo resultado siempre)	Dev	
☐	Pool de conexiones: no hay connection leaks después de cada test	Dev	

4.3 Pruebas de Base de Datos NoSQL (MongoDB/Redis) - Cumplir con al menos 2 puntos

	Ítem	Responsable	Estado
☐	Testcontainers o mongodb-memory-server para MongoDB en tests	Dev	
☐	Colecciones limpias entre tests (deleteMany o recrear DB de test)	Dev	
☐	Tests de documentos: validación de schema, campos opcionales, arrays anidados	Dev	
☐	Índices de MongoDB: consultas con explain() no hacen collection scan	Dev	
☐	Aggregation pipelines probadas con datasets representativos	Dev	
☐	TTL de documentos (si aplica): expiración correcta de registros	Dev	
☐	Redis (si aplica): set, get, expire, invalidación de caché correcta	Dev	
☐	Consistencia eventual: si hay replicación, los tests contemplan el delay	Dev	

5

Pruebas End-to-End

Cypress / Playwright · Flujos completos desde el navegador

5.1 Selección de flujos a automatizar - Cumplir con todos los puntos

Flujos críticos que SIEMPRE deben tener E2E:

	Ítem	Responsable	Estado
☐	Registro de usuario nuevo (incluyendo validaciones del formulario)	QA	
☐	Login / Logout con credenciales válidas e inválidas	QA	
☐	Flujo principal del negocio de extremo a extremo (el core del producto)	QA	
☐	Flujo de pago o transacción crítica (si aplica al proyecto)	QA	
☐	Recuperación de contraseña / gestión de sesión	QA	
☐	Roles y permisos: admin vs usuario regular vs visitante	QA	
☐	Flujos de error críticos: ¿qué ve el usuario cuando el servidor falla?	QA	
☐	Formularios complejos con múltiples pasos (wizards)	QA	

5.2 Buenas prácticas de E2E con Cypress/Playwright

	Ítem	Responsable	Estado
☐	Todos los selectores usan data-testid o data-cy — nunca clases CSS	QA/Dev	
☐	Custom commands para acciones repetidas (loginAs, crearRecurso, etc.)	QA	
☐	beforeEach hace login via API directamente (no repitiendo el flujo de UI)	QA	
☐	cy.intercept() o page.route() para controlar respuestas de red	QA	
☐	Fixtures para datos de prueba consistentes entre tests	QA	
☐	Tests independientes: sin dependencia del orden ni de tests anteriores	QA	
☐	Limpieza de BD entre tests (endpoint de reset o seeders dedicados)	Dev	
☐	Screenshots y videos activados en el pipeline para análisis de fallos	DevOps	
☐	Retry de tests inestables con límite de 2 reintentos	QA	
☐	Tests agrupados por módulo de negocio, no por tipo de acción	QA	
☐	Page Object Model o App Actions para encapsular interacciones de UI	QA	
☐	Verificación de accesibilidad básica incluida en los E2E críticos	QA	

6

Pruebas de Rendimiento

k6 / Artillery · Load · Stress · Spike · Soak

6.2 Tipos de prueba de rendimiento - Cumplir con todos los puntos

	Ítem	Responsable	Estado
☐	Load Test: usuarios esperados en producción por 10 minutos con ramp-up de 5 min	QA/DevOps	
☐	Stress Test: aumentar usuarios hasta encontrar el punto de quiebre del sistema	QA/DevOps	
☐	Spike Test: pico repentino de 10x el tráfico normal por 1 minuto	QA/DevOps	
☐	Soak Test (Endurance): carga normal por 2 horas para detectar memory leaks	QA/DevOps	

7

Pruebas de Seguridad

OWASP ZAP · Snyk · Checklist OWASP Top 10

7.1 OWASP Top 10 — verificación por vulnerabilidad - Cumplir con al menos 3 puntos

	Ítem	Responsable	Estado
☐	A01 — Broken Access Control: endpoints protegidos solo por roles correctos	Security/QA	
☐	A01 — Tests de autorización: usuario A no puede ver/modificar datos de usuario B	Security/QA	
☐	A02 — Cryptographic Failures: passwords hasheados con bcrypt/argon2 (nunca MD5/SHA1)	Dev	
☐	A02 — Datos sensibles no aparecen en logs ni en respuestas de error	Dev	
☐	A03 — Injection SQL: todos los queries usan prepared statements o ORM	Dev	
☐	A03 — Injection NoSQL: inputs sanitizados antes de queries en MongoDB	Dev	
☐	A03 — XSS: outputs de datos del usuario son escapados en el frontend	Dev	
☐	A04 — Insecure Design: threat modeling documentado para flujos críticos	Arquitecto	
☐	A05 — Security Misconfiguration: headers de seguridad presentes (helmet.js)	Dev	
☐	A05 — CORS configurado restrictivamente (no wildcard * en producción)	Dev	
☐	A05 — Mensajes de error genéricos en producción (sin stack traces)	Dev	
☐	A06 — Vulnerable Components: Snyk o npm audit en el pipeline de CI	DevOps	
☐	A06 — Dependencias actualizadas, sin CVEs de severidad crítica o alta	Dev	
☐	A07 — Auth Failures: rate limiting en login (máx 5 intentos por IP/minuto)	Dev	
☐	A07 — Tokens JWT con expiración, algoritmo seguro (RS256 o HS256 mínimo)	Dev	
☐	A07 — Refresh tokens con rotación y revocación implementada	Dev	
☐	A08 — Software Integrity: dependencias con checksums verificados (lockfile)	Dev	
☐	A09 — Logging: todas las operaciones críticas logeadas con audit trail	Dev	
☐	A09 — Alertas configuradas para anomalías de seguridad	DevOps	

8

Accesibilidad y Compatibilidad

WCAG 2.1 · axe-core · Lighthouse · Cross-browser

8.1 Accesibilidad — WCAG 2.1 - Cumplir con todos los puntos

	Ítem	Responsable	Estado
☐	Todas las imágenes tienen atributo alt descriptivo	Dev	
☐	El sitio es completamente navegable solo con teclado (Tab, Enter, Esc, flechas)	QA	
☐	Focus visible en todos los elementos interactivos	Dev	
☐	Formularios: todos los inputs tienen label asociado o aria-label	Dev	
☐	Errores de formulario comunicados con aria-describedby o aria-live	Dev	

8.2 Compatibilidad de navegadores y dispositivos - Cumplir con al menos 6 puntos

	Ítem	Responsable	Estado
☐	Chrome: última versión y última-1	QA	
☐	Firefox: última versión	QA	
☐	Safari: última versión en macOS e iOS	QA	
☐	Edge: última versión	QA	
☐	Viewport desktop: 1920×1080, 1440×900, 1280×720	QA	
☐	Viewport tablet: 768×1024 (iPad)	QA	
☐	Viewport móvil: 390×844 (iPhone 14), 360×800 (Android típico)	QA	
☐	Tests de Cypress/Playwright ejecutados en múltiples viewports	QA	
☐	Prueba con JavaScript desactivado: contenido crítico accesible	Dev	

11

Monitoreo en Producción y Observabilidad

Logs · Métricas · Alertas · Error Tracking

11.1 Los tres pilares de la observabilidad

Logs - Cumplir con todos los puntos

	Ítem	Responsable	Estado
☐	Logging estructurado en JSON (Pino, Winston) para facilitar búsqueda	Dev	
☐	Niveles de log correctos: error, warn, info, debug (no todo en info)	Dev	
☐	Correlation ID en cada request para trazabilidad end-to-end	Dev	
☐	Sin datos sensibles en logs (passwords, tokens, PII)	Dev	
☐	Reporte de logs	DevOps	
☐	Retención de logs definida (ej: 30 días hot, 90 días cold)	DevOps	

Métricas - Cumplir con al menos 2 puntos

	Ítem	Responsable	Estado
☐	RED metrics por endpoint: Rate (req/s), Errors (%), Duration (ms)	Dev/DevOps	
☐	USE metrics de infraestructura: Utilization, Saturation, Errors	DevOps	
☐	Business metrics: conversiones, registros, transacciones por minuto	Dev	
☐	Dashboard de métricas en tiempo real (Grafana, Datadog)	DevOps	
☐	Métricas de BD: conexiones activas, query duration, deadlocks	DevOps	